

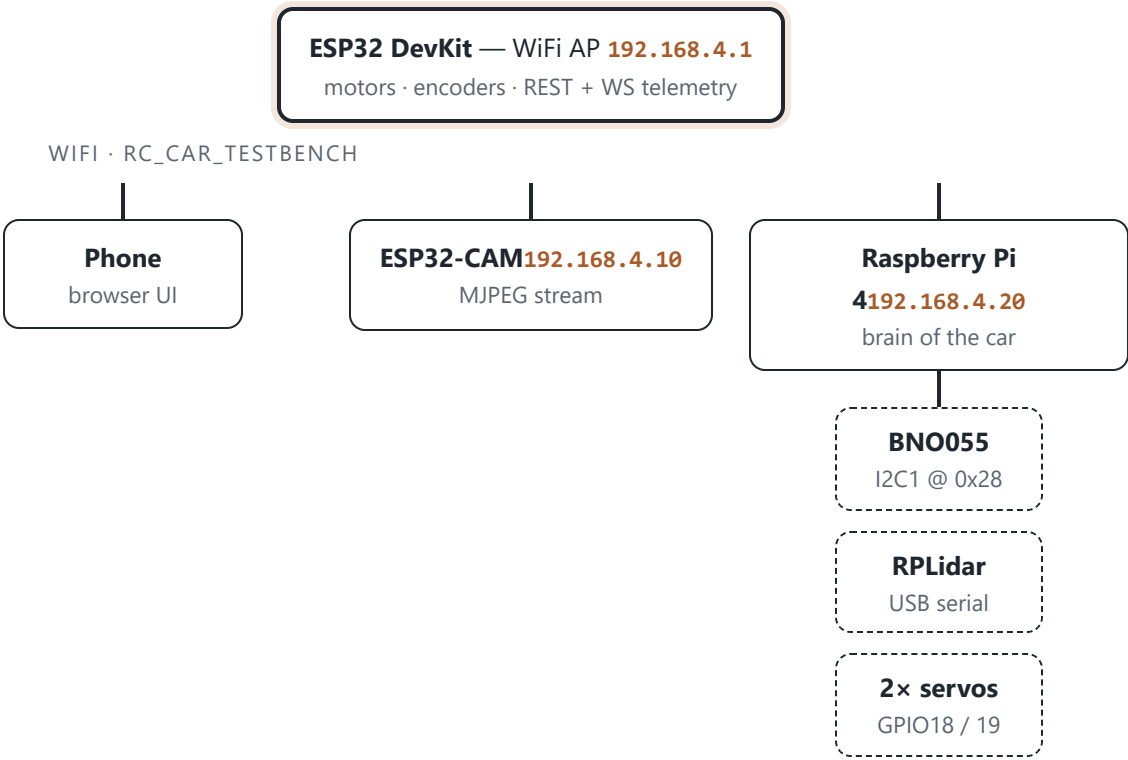
Raspberry Pi Integration Phase

Status **planning** / wiring reference Boards **Pi 4 · ESP32 DevKit · ESP32-CAM**

New hardware **BNO055 · RPLidar A1/C1 · 2× gimbal servos** Rev July 2026

System topology

Three compute boards, one WiFi network, no new wires between boards. The ESP32 drive firmware (`CarTestBench/`) and the CAM streamer (`CamStreamer/`) are unchanged by this phase.



- The Pi is a WiFi **station** on the ESP32's AP, exactly like the phone.
- It drives the car through the existing REST API (`/api/drive` , `/api/motor`) and reads wheel telemetry from the WebSocket (`/ws`).
- It reads video straight from the CAM at `http://192.168.4.10:81/stream` .
- IMU, LiDAR and the gimbal servos wire to the **Pi only** — nothing new touches the ESP32 pin map (it has no spare GPIO anyway).

Network plan

DEVICE	IP	HOW
ESP32 DevKit (AP)	192.168.4.1	fixed by softAP
ESP32-CAM	192.168.4.10	static in <code>CamStreamer.ino</code>
Raspberry Pi 4	192.168.4.20	static, set with NetworkManager
Phone	DHCP	whatever the AP assigns

The softAP allows 4 stations by default: CAM + Pi + phone fits. Later, when the Pi permanently lives on the car, the AP role can move to the Pi (hostapd) and the two ESP32s become stations — same topology, better radio.

Raspberry Pi 4 pinout

Physical header pin numbers in parentheses.

BNO055 IMU — I2C1

BNO055 PIN	PI PIN	NOTES
VIN	3V3 (pin 1)	3.3 V only
GND	GND (pin 6)	
SDA	GPI02 / SDA1 (pin 3)	
SCL	GPI03 / SCL1 (pin 5)	
ADR	GND or open	open = address <code>0x28</code> (matches the old firmware constant)

Known gotcha. The BNO055 uses I2C clock stretching and the Pi's hardware I2C handles it badly. Slow the bus in `/boot/firmware/config.txt` :

```
dtparam=i2c_arm=on
dtparam=i2c_arm_baudrate=10000
```

10 kHz is plenty for 100 Hz orientation reads. Verify with `i2cdetect -y 1` — expect a device at `0x28` .

Mount the IMU rigidly near the car's center, away from the motors (magnetometer distortion), on a small foam pad for vibration isolation. Note the axis orientation you mount it in — it goes into the fusion config.

Gimbal servos — hardware PWM

SERVO	SIGNAL	PI PIN	POWER
Pan	GPI018 (PWM0)	pin 12	external 5 V UBEC
Tilt	GPI019 (PWM1)	pin 35	external 5 V UBEC
Both	GND	any GND + UBEC GND	common ground required

- **Never power servos from the Pi's 5 V pins.** A stalled MG90S/MG995 pulls over 1 A and will brown out the Pi. Use a dedicated 5 V UBEC (≥ 3 A) from the drive battery, ground tied to Pi ground.
- Standard hobby PWM: 50 Hz, 1.0–2.0 ms pulse. Use `pigpio` (hardware-timed) — never software PWM, it jitters.
- Pi logic is 3.3 V; hobby servos on 5 V power accept a 3.3 V signal fine. If a servo twitches or misses pulses, add a level shifter on the two signal lines.
- `GPI012` (pin 32) and `GPI013` (pin 33) are the alternate hardware-PWM pins if 18/19 are ever needed for something else.

RPLidar A1 / C1 — USB

ITEM	VALUE
Connection	bundled USB-serial adapter → any Pi USB-A port
Device	<code>/dev/ttyUSB0</code>
Baud	A1: 115200 · C1: 460800
Power	from the USB adapter (5 V, ~0.5 A incl. spin motor)

No GPIO used. The Pi 4's USB ports supply 1.2 A total across all ports — enough for the LiDAR, but power the Pi itself properly (next section). Give the LiDAR an unobstructed 360° view; mount it as the highest thing on the chassis.

Pins left free

I2C0 (27/28), all SPI pins, UART0 (8/10), and everything else on the 40-pin header stays free for the ToF sensor and future additions.

Power architecture

RAIL	SOURCE	FEEDS	SIZING
Drive battery	main pack	TB6612 VM (motors)	existing
5 V #1	buck/UBEC ≥ 3 A (5 A ideal)	Pi 4 via USB-C	Pi 4 wants 3 A; LiDAR adds ~ 0.5 A through Pi USB
5 V #2	UBEC ≥ 3 A	both gimbal servos	stall spikes — keep separate from the Pi
5 V #3	existing supply	ESP32 DevKit + ESP32-CAM	CAM needs solid 2 A headroom
3.3 V	Pi's own pin 1	BNO055 only	mA-level

All grounds tie together at one star point (battery negative / main ground bus).
WiFi boards + I2C + servo PWM only work reliably with a common ground.

Boot order in practice: ESP32 DevKit first (brings up the AP), then CAM and Pi in any order — both retry until the AP appears.

Pi ↔ car interfaces (software)

PEER	TRANSPORT	INTERFACE
ESP32 DevKit	WiFi HTTP	POST /api/drive · /api/motor · /api/estop
ESP32 DevKit	WiFi WS	ws://192.168.4.1/ws — 10 Hz JSON: pwm, armed, encoder counts/RPM
ESP32-CAM	WiFi HTTP	:81/stream · /capture · /control · /status
BNO055	I2C1 @ 0x28	<code>adafruit-circuitpython-bno055</code>
RPLidar	USB serial	<code>rplidar-roboticia</code> (Slamtec SDK / ROS 2 later)
Servos	GPIO18/19 PWM	<code>pigpio</code>

Pi software checklist

1. Raspberry Pi OS 64-bit (Bookworm). Enable I2C: `sudo raspi-config` → Interface Options, plus the `config.txt` baudrate lines above.
2. Join the car AP with a static IP:

```
sudo nmcli c add type wifi ifname wlan0 con-name car ssid RC_Car_TestBer
sudo nmcli c modify car wifi-sec.key-mgmt wpa-psk wifi-sec.psk carbench1
    ipv4.method manual ipv4.addresses 192.168.4.20/24 ipv4.gateway 192.
sudo nmcli c up car
```

While on the car AP the Pi has no internet — install packages on home WiFi first, or keep two NetworkManager profiles.

3. Packages: `sudo apt install python3-pip i2c-tools pigpio python3-pigpio`
then `sudo systemctl enable --now pigpiod`.

4. Python libs:

```
pip install adafruit-circuitpython-bno055 rplidar-roboticia opencv-python websocket-client requests
```

5. Smoke tests, in order:

- `i2cdetect -y 1` → `0x28` appears.
- `ping 192.168.4.1`, then `curl -X POST "http://192.168.4.1/api/estop"` → `ESTOP` (proves the control path; clear it from the phone UI).
- `python -c "import cv2;c=cv2.VideoCapture('http://192.168.4.10:81/stream');print(c.read()[0])"` → `True`.
- RPLidar: the 5-line read loop from the `rplidar` package docs prints scans.
- Servos: pigpio `set_servo_pulsewidth(18, 1500)` centers the pan servo.

Safety notes

- The E-stop chain is unchanged: phone UI and Pi both hit `POST /api/estop`; STBY pull-downs still cut the bridges in hardware. Any Pi autonomy script should send `/api/estop` on exception/exit.
- Keep the drive endpoint's deadman pattern: the Pi should re-send its drive command periodically and stop the car if its control loop stalls.
- Servos, the LiDAR motor and the CAM all spike current: if the Pi ever reboots when the gimbal moves, the rails are undersized or shared.
- 3.3 V logic only on Pi GPIO. The BNO055 breakout must be a 3.3 V-capable board (Adafruit and most clones are).